



Ampliación de Sitemas Operativos

Grado en Ingeniería Informática

Curso 2025/26

Miguel Ángel Marín Valero

miguelangel.marinv@um.es

Grupo 1.2

Pedro Rubio Albarracín

pedro.rubioa@um.es

Grupo 1.2

Índice

1. Boletín 1: Llamadas al sistema	2
1.1. Ejercicio 1: llamada date	2
1.2. Ejercicio 2: llamada dup2	3
1.3. Ejercicio 3: llamada exit and wait	5
1.4. Test	7
2. Boletín 2: Sistema de memoria. Reserva de páginas bajo demanda	8
2.1. Ejercicio 1	8
2.2. Ejercicio 2	10
2.3. Test	11
3. Boletín 3: Planificación de procesos y procesos de prioridad alta	12
3.1. Ejercicio 1	12
3.2. Ejercicio 2	16
3.3. Test	18

Boletín 1: Llamadas al sistema

1.1. Ejercicio 1: llamada date

```
23 #define SYS_date 22
```

Figura 1: Fichero syscall.h

```
33 SYSCALL(date)
```

Figura 2: Fichero usys.S

```
36 extern int date(struct rtcdate*);
```

Figura 3: Fichero user.h

```
106 extern int sys_date(void);
```

Figura 4: Fichero syscall.c

```
131 [SYS_date] sys_date,
```

Figura 5: Fichero syscall.c

```
int
sys_date(void)
{
    struct rtcdate *r;

    if(argptr(0, (void **)&r, sizeof(struct rtcdate)) < 0){
        return -1;
    }
    cmostime(r);

    return 0;
}
```

Figura 6: Fichero sysproc.c

1.2. Ejercicio 2: llamada dup2

```
24  #define SYS_dup2 23
```

Figura 7: Fichero syscall.h

```
33  SYSCALL(dup2)
```

Figura 8: Fichero usys.S

```
36  extern int dup2(int, int);
```

Figura 9: Fichero user.h

```
107 extern int sys_dup2(void);
```

Figura 10: Fichero syscall.c

```
32  [SYS_dup2] sys_dup2,
```

Figura 11: Fichero syscall.c

```

int
sys_dup2(void)
{
    struct file *f, *nf;
    int oldfd, newfd;

    if(argint(0, &oldfd) < 0 || argint(1, &newfd) < 0){
        return -1;
    }

    if(oldfd < 0 || oldfd >= NOFILE || (f=myproc()->ofile[oldfd]) == 0){
        return -1;
    }

    if(newfd < 0 || newfd >= NOFILE){
        return -1;
    }

    if(oldfd == newfd){
        return newfd;
    }

    nf = myproc()->ofile[newfd];
    if(nf){
        fileclose(nf);
    }

    myproc()->ofile[newfd] = f;
    filedup(f);
    return newfd;
}

```

Figura 12: Fichero sysfile.c

```

100     case PIPE:
101         pcmd = (struct pipecmd*)cmd;
102         if(pipe(p) < 0)
103             panic("pipe");
104         if(fork1() == 0){
105             //
106             dup2(p[1], 1);
107             close(p[0]);
108             close(p[1]);
109             runcmd(pcmd->left);
110         }
111         if(fork1() == 0){
112             //
113             dup2(p[0], 0);
114             close(p[0]);
115             close(p[1]);
116             runcmd(pcmd->right);
117         }
118         close(p[0]);
119         close(p[1]);
120         wait(NULL);
121         wait(NULL);
122         break;

```

Figura 13: Fichero sh.c

1.3. Ejercicio 3: llamada exit and wait

```
120 int wait(int* status);
```

Figura 14: Fichero defs.h

```
107 void exit(int e_status);
```

Figura 15: Fichero defs.h

```
16 int
17 sys_exit(void)
18 {
19     int e_status;
20
21     if(argint(0, &e_status) < 0){
22         return -1;
23     }
24     e_status = e_status << 8;
25     exit(e_status);
26     return 0;
27 }
28
29 int
30 sys_wait(void)
31 {
32     int *status;
33
34     if(argptr(0, (void**)&status, sizeof(int)) < 0){
35         return -1;
36     }
37
38     return wait(status);
39 }
```

Figura 16: Fichero sysproc.c

```
7 #define WIFEXITED(status) (((status) & 0x7f) == 0)
8 #define WEXITSTATUS(status) (((status) & 0xff00 ) >> 8)
9 #define WIFSIGNALED(status) (((status) & 0x7f ) != 0)
10 #define WEXITTRAP(status) (((status) & 0x7f ) - 1)
```

Figura 17: Fichero user.h

```
15 extern int exit(int e_status) __attribute__((noreturn));
16 extern int wait(int* status);
```

Figura 18: Fichero user.h

```
52 int status;
```

Figura 19: Fichero proc.h

```

227 void
228 exit(int e_status)
229 {

```

Figura 20: Fichero proc.c

```

// Jump into the scheduler, never to return.
curproc->state = ZOMBIE;
curproc->status = e_status;

```

Figura 21: Fichero proc.c

```

279 int
280 wait(int *status)

```

Figura 22: Fichero proc.c

```

170 int status;
171 if(wait(&status) <= 0)
172     continue;
173
174 if(WIFEXITED(status)){
175     printf(1, "Output Code: %d\n", WEXITSTATUS(status));
176
177 } else if(WIFSIGNALED(status)){
178     printf(1, "Output Code: %d\n", WEXITSTATUS(status));
179 }
180
181 }
182 exit(0);
183 }

```

Figura 23: Fichero sh.c

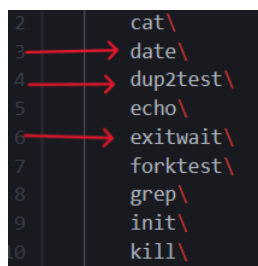
```

void
trap(struct trapframe *tf)
{
    if(tf->trapno == T_SYSCALL){
        if(myproc()->killed)
            exit(tf->trapno+1);
        myproc()->tf = tf;
        syscall();
        if(myproc()->killed)
            exit(tf->trapno+1);
        return;
    }
}

```

Figura 24: Fichero trap.c

1.4. Test



```
2: cat\  
3: date\  
4: dup2test\  
5: echo\  
6: exitwait\  
7: forktest\  
8: grep\  
9: init\  
10: kill\
```

Figura 25: Makefile de usuario.

Boletín 2: Sistema de memoria. Reserva de páginas bajo demanda

2.1. Ejercicio 1

```
88 int
89 sys_sbrk(void)
90 {
91     int addr;
92     int n;
93
94     if(argint(0, &n) < 0)
95         return -1;
96
97     addr = myproc()->sz;
98
99     if(n<0)
100     {
101         if (growproc(n) < 0)
102             return -1;
103     }
104     else
105     {
106         myproc()->sz += n;
107     }
108
109     return addr;
110 }
111
112
```

Figura 26: Fichero sysproc.c

```
101 curproc->tf->esp = sp;
102 curproc->userPage = sz - PGSIZE; //guardar la dirección de la página de usuario a
103
```

Figura 27: Fichero exec.c

```
52 int status;
53 uint userPage;
```

Figura 28: Fichero proc.h

```

145     case T_PGFLT: {
146         uint va = rcr2();
147         char *mem;
148
149         if(va >= KERNBASE || va >= myproc()->sz || va < myproc()->userPage) {
150             cprintf("Page fault: Acceso a memoria invalida (0x%x)\n", va);
151             myproc()->killed = 1;
152             break;
153         }
154         va = PGROUNDDOWN(va);
155
156         mem = kalloc();
157         if(mem == 0) {
158             cprintf("T_PGFLT: kalloc out of memory\n");
159             myproc()->killed = 1;
160             break;
161         }
162
163         memset(mem, 0, PGSIZE);
164
165         if(mappages(myproc()->pgdir, (char*)va, PGSIZE, V2P(mem), PTE_W|PTE_U) < 0) {
166             cprintf("T_PGFLT: mappages failed\n");
167             kfree(mem);
168             myproc()->killed = 1;
169             break;
170         }
171
172         break;
173     }

```

Figura 29: Fichero trap.c

```

61 //se borra la palabra static( ya declarada externamente en trap.c)
62 int
63 mappages(pde_t *pgdir, void *va, uint size, uint pa, int perm)
64 {
65     char *a, *last;
66     pte_t *pte;
67
68     a = (char*)PGROUNDDOWN((uint)va);

```

Figura 30: Fichero vm.c

2.2. Ejercicio 2

```
145 case T_PGFLT: {
146     uint va = rcr2();
147     char *mem;
148
149     if(va >= KERNBASE || va >= myproc()->sz || va < myproc()->userPage) {
150         cprintf("Page fault: Acceso a memoria invalida (0x%x)\n", va);
151         myproc()->killed = 1;
152         break;
153     }
154     va = PGROUNDDOWN(va);
155
156     mem = kalloc();
157     if(mem == 0) {
158         cprintf("T_PGFLT: kalloc out of memory\n");
159         myproc()->killed = 1;
160         break;
161     }
162
163     memset(mem, 0, PGSIZE);
164
165     if(mappages(myproc()->pgdir, (char*)va, PGSIZE, V2P(mem), PTE_W|PTE_U) < 0) {
166         cprintf("T_PGFLT: mappages failed\n");
167         kfree(mem);
168         myproc()->killed = 1;
169         break;
170     }
171
172     break;
173 }
```

Figura 31: Fichero trap.c

```
215 // until it gets to the regular system call return.)
216 if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER)
217     exit(tf->trapno+1);
218
219 // Force process to give up CPU on clock tick.
220 // If interrupts were on while locks held, would need to check
221 if(myproc() && myproc()->state == RUNNING &&
222     tf->trapno == T_IRQ0+IRQ_TIMER)
223     yield();
224
225 // Check if the process has been killed since we yielded
226 if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER)
227     exit(tf->trapno+1);
228 }
229
```

Figura 32: Fichero trap.c

```

35 //se borra la palabra static( ya declarada externamente en trap.c)
36 → pte_t *
37 walkpgdir(pde_t *pgdir, const void *va, int alloc)
38 {
39     pde_t *pde;
40     pte_t *pgtab;
41
42     pde = &pgdir[PDX(va)];

```

Figura 33: Fichero vm.c

2.3. Test

```

16 stressfs\
17 usertests\
18 + wc\
19 zombie\
20 { tsbrk1\
21   tsbrk2\
22   tsbrk3\
23   tsbrk4\
24   tsbrk5\
25 | tprio\

```

Figura 34: Makefile de usuario. tsbrk

Boletín 3: Planificación de procesos y procesos de prioridad alta

3.1. Ejercicio 1

```
14  
15 #define MAXPRIO 10 // Prioridad máxima  
16
```

Figura 35: Fichero param.h

```
uint prio; // Prioridad del proceso  
struct proc *siguiente; // Siguiente proceso en la cola
```

Figura 36: Fichero proc.h

```
curproc->prio = 5; //prioridad del proceso al valor máximo  
curproc->siguiente = NULL; //puntero al siguiente proceso en la cola como nulo
```

Figura 37: Fichero exec.c

```
struct cola { // Estructura para guardar primer y ultimo proceso de la cola  
    struct proc *primer;  
    struct proc *ultimo;  
};  
  
struct {  
    struct spinlock lock;  
    struct proc proc[NPROC];  
    struct cola tabla[MAXPRIO]; //tabla de colas  
} ptable;
```

Figura 38: Fichero proc.c

```

//Funcion para añadir proceso al final de cola
void
anadeProceso(struct proc *p)
{
    struct cola *c = &ptable.tabla[p->prio];
    if(c->primer != NULL)
    {
        struct proc *penultimo = c->ultimo;
        c->ultimo = p;
        penultimo->siguiente = p;
    }
    else
    {
        c->primer = p;
        c->ultimo = p;
    }
    p->siguiente = NULL;
}

```

Figura 39: Fichero proc.c . Funcion anadeProceso()

```

8 //Funcion para eliminar el proceso al comienzo de cola
9 void
10 quitaProceso(struct proc *p)
11 {
12     struct cola *c = &ptable.tabla[p->prio];
13     struct proc *ini = c->primer;
14     if(ini == NULL)
15         return;
16
17     if(ini->siguiente == NULL)
18     {
19         c->primer = NULL;
20         c->ultimo = NULL;
21         return;
22     }
23
24     c->primer = ini->siguiente;
25     ini->siguiente = NULL;
26
27 }

```

Figura 40: Fichero proc.c . Funcion quitaProceso()

```

203 // because the assignment might not be atomic
204 acquire(&ptable.lock);
205
206 p->state = RUNNABLE;
207 anadeProceso(p); // Añadir proceso a la cola
208
209 release(&ptable.lock);
210 }

```

Figura 41: Fichero proc.c . Funcion userinit()

```

267     safestrcpy(np->name, curproc->name, sizeof(curproc->name));
268     np->prio = curproc->prio; // El proceso hijo hereda la prior
269
270     pid = np->pid;
271
272     acquire(&ptable.lock);
273
274     np->state = RUNNABLE;
275
276     anadeProceso(np); // Añadir proceso a la cola de prioridades
277
278     release(&ptable.lock);
279
280     return pid;
281
282 }

```

Figura 42: Fichero proc.c . Funcion fork()

```

uint i = 0;
while (i < MAXPRIO) { //Recorremos todas las colas de prioridades
    struct cola *colaActual = &ptable.tabla[i]; //Se obtiene la cola en fu
    p = colaActual->primer; //Obtenemos el primer elemento de la cola
    if (p == NULL) { //Si la cola está vacía, pasamos a la siguiente
        i++;
        continue;
    }
    //Si hay un proceso en la cola, lo sacamos de la cola para ejecutarlo
    quitaProceso(p);

    // Switch to chosen process. It is the process's job
    // to release ptable.lock and then reacquire it
    // before jumping back to us.
    c->proc = p;
    switchvm(p);
    p->state = RUNNING;

    swtch(&(c->scheduler), p->context);
    switchkvm();

    // Process is done running for now.
    // It should have changed its p->state before coming back.
    c->proc = 0;
    //Se finaliza el planificador ya que se ha elegido un proceso para eje
    i = MAXPRIO;
}
release(&ptable.lock);

```

Figura 43: Fichero proc.c . Funcion scheduler()

```

471 // Give up the CPU for one scheduling round.
472 void
473 yield(void)
474 {
475     acquire(&ptable.lock); //DOC: yieldlock
476     myproc()->state = RUNNABLE;
477     anadeProceso(myproc()); // Volver a añadir el proceso a la cola de prioridades
478     sched();
479     release(&ptable.lock);
480 }

```

Figura 44: Fichero proc.c . Funcion yield()

```

545 static void
546 wakeup1(void *chan)
547 {
548     struct proc *p;
549
550     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
551         if(p->state == SLEEPING && p->chan == chan) {
552             p->state = RUNNABLE;
553             anadeProceso(p); //Al pasar a listo, volver a añadir
554         }
555 }
556

```

Figura 45: Fichero proc.c . Funcion wakeup1()

```

569 int
570 kill(int pid)
571 {
572     struct proc *p;
573
574     acquire(&ptable.lock);
575     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
576         if(p->pid == pid){
577             p->killed = 1;
578             // Wake process from sleep if necessary.
579             if(p->state == SLEEPING){
580                 p->state = RUNNABLE;
581                 anadeProceso(p); //Al pasar a listo, volver a
582             }
583             release(&ptable.lock);
584             return 0;
585         }
586     }
587     release(&ptable.lock);
588     return -1;
589 }
590

```

Figura 46: Fichero proc.c . Funcion kill()

3.2. Ejercicio 2

```
27 SYSCALL(dup)
28 SYSCALL(getpid)
29 SYSCALL(sbrk)
30 SYSCALL(sleep)
31 SYSCALL(uptime)
32 SYSCALL(date)
33 SYSCALL(dup2)
34
35 } SYSCALL(getprio)
36 SYSCALL(setprio)
37 +
```

Figura 47: Fichero Usys.s

```
34 extern int uptime(void);
35 extern int date(struct rtcdate*);
36 extern int dup2(int, int);
37
38 } extern int getprio(int pid);
39 extern int setprio(int pid, unsigned int prio);
40
41 // ulib.c
```

Figura 48: Fichero user.h

```
119 void userinit(void);
120 int wait(int* status);
121 void wakeup(void*);
122 void yield(void);
123
124 } int getprio(int);
125 int setprio(int, unsigned int);
126
```

Figura 49: Fichero defs.h

```
22 #define SYS_close 21
23 #define SYS_date 22
24 #define SYS_dup2 23
25
26 } #define SYS_getprio 24
27 #define SYS_setprio 25
28
```

Figura 50: Fichero syscall.h

```
106 extern int sys_date(void);
107 extern int sys_dup2(void);
108
109 } extern int sys_getprio(void);
110 extern int sys_setprio(void);
```

Figura 51: Fichero syscall.c

```

129     [SYS_unlink] sys_unlink,
130     [SYS_link]   sys_link,
131     [SYS_mkdir]  sys_mkdir,
132     [SYS_close]  sys_close,
133     [SYS_date]   sys_date,
134     [SYS_dup2]   sys_dup2,
135 +
136     [SYS_getprio] sys_getprio,
137     [SYS_setprio] sys_setprio,
138 };

```

Figura 52: Fichero syscall.c

```

56
57
58     int
59     sys_getprio(void)
60     {
61         int pid;
62         if(argint(0, &pid) < 0)
63             return -1;
64         return getprio(pid);
65     }
66
67
68
69     int
70     sys_setprio(void)
71     {
72         int pid;
73         int aux;
74
75         if(argint(0, &pid) < 0)
76             return -1;
77
78         if(argint(1, &aux) < 0)
79             return -1;
80
81         unsigned int prio = (unsigned int) aux;
82
83         return setprio(pid, prio);
84     }

```

Figura 53: Fichero sysproc.c . Function sys_getprio() y sys_setprio()

```

10 #define MAXOPBLOCKS 10 // max # of blocks any FS op writes
11 #define LOGSIZE      (MAXOPBLOCKS*3) // max data blocks in on-disk log
12 #define NBUF         (MAXOPBLOCKS*3) // size of disk block cache
13 #define FSSIZE       1000 // size of file system in blocks
14 #define MAXPRIO 10 // Numero de prioridades(0 a 9)
15 #define LOWEST_PRIO 9
16 #define NORM_PRIO 5
17 #define HIGHEST_PRIO 0
18

```

Figura 54: Fichero param.h

```

628
629 int
630 getprio(int pid)
631 {
632     if(pid <= 0)
633         return -1;
634
635     struct proc *p;
636     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
637         if(p->pid == pid){
638             return p->prio;
639         }
640     }
641
642     return -1;
643 }
644
645
646 int
647 setprio(int pid, unsigned int prio)
648 {
649     if(pid <= 0 || prio >= MAXPRIO || prio < 0)
650         return -1;
651
652     struct proc *p;
653     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
654         if(p->pid == pid){
655             p->prio = prio;
656             return 0;
657         }
658     }
659
660     return -1;
661 }

```

Figura 55: Fichero proc.c . Function getprio() y setprio()

3.3. Test

```

19 + zombie\
20 tsbrk1\
21 tsbrk2\
22 tsbrk3\
23 tsbrk4\
24 tsbrk5\
25 → tprio\

```

Figura 56: Makefile de usuario. tprio

```

188     else echo "3 cp"
189     ifndef CPUS
190 → CPUS := 1
191     endif

```

Figura 57: Fichero Makefile

```

1  #include "types.h"
2  #include "user.h"
3  }
4 → #include "param.h"
5

```

Figura 58: Fichero tprio

```

22
23
24 int
25 main(int argc, char *argv[])
26 {
27 → exit(0);
28
29 // El proceso se inicia en baja prioridad.
30 // Genera otro proceso hijo que a su vez genera dos
31 printf(1, "Hay 4 procesos de minima prioridad ejecutandose, asi que deberian verse intercalados.\n");
32
33 if (fork() == 0)
34 {
35     if (fork()) // Ambos ejecutan:
36     { setprio(getpid(), LOWEST_PRIO); do_calc("-"); }
37     else
38 + { setprio(getpid(), LOWEST_PRIO); do_calc("+"); }
39
40 → exit(0);
41 }
42
43 if (fork() == 0)
44 {
45     if (fork()) // Ambos ejecutan:
46     { setprio(getpid(), NORM_PRIO); do_calc("*"); }
47     else
48     { setprio(getpid(), LOWEST_PRIO); do_calc("^"); }
49
50 → exit(0);
51 }
52

```

Figura 59: Fichero tprio

```

52
53 printf(1, "Me voy a dormir 10 segundos para que puedas interactuar con el shell. \
54 Como el shell tiene más prioridad que estos dos procesos, imprimir sin ser interrumpido por ellos.\n");
55 sleep(500);
56
57
58 printf(1, "Y ahora se lanzan dos de alta prioridad. Deberían mostrarse las prioridades \
59 en orden creciente conforme acaban los procesos, y el shell no debería tener interacción.\n");
60 printf(1, "Cuando terminen los dos de alta prioridad, deberían seguir los de baja hasta terminar.\n");
61 if (fork() == 0)
62 {
63     if (fork()) // Ambos ejecutan
64     {
65         setprio (getpid(), HIGHEST_PRIO);
66         do_calc("0");
67         exit(0);
68     }
69     else
70     {
71         setprio (getpid(), HIGHEST_PRIO+1);
72         do_calc("1");
73         exit(0);}
74     }
75
76     exit(0);
77 }

```

Figura 60: Fichero tprio